Haystack Dependency Injection

Runtime and editor guide for the RD.Framework Haystack package.

Direct Injection | Indirect Builder | Collider Injection

Author: Reckdev

Benefits of Using Haystack

Cleaner Architecture

Dependencies are declared with attributes and installed in one place, reducing tight coupling and making gameplay systems easier to maintain.

Faster Iteration

Direct, indirect, and manual binding flows let you adapt quickly to scene-based or runtime-generated objects without rewriting core systems.

Cross-Platform Friendly

Coroutine-based setup support avoids `async` / `await` pitfalls on WebGL while keeping behavior consistent on desktop and mobile.

Debuggable Runtime

Try-resolve, explicit unbind, and targeted key slots help isolate dependency issues during play mode with predictable results.

Gameplay-Aware Injection

Collider and Collider2D dependency paths enable interaction-driven DI patterns for pickup, overlap, trigger, and combat systems.

Package-Ready Workflow

Editor inspectors, runtime config assets, and protected core/runtime scripts keep your framework organized and easier to ship as a package.

Core API

Attributes

- `[Inject]` injects a registered dependency.
- `[Install]` registers a dependency.
- `[HayDirectClass]` opts a class into scene-scan install and inject passes.
- `[HayPhysicsClass]` enables collider-based dependency pickup.
- `[HayInstallingMethod]` runs before installation and may return `IEnumerator` for coroutine-based setup.
- `[HayInjectedMethod]` runs after injection and may return `IEnumerator` for coroutine-based setup.
- Prefer `IEnumerator` coroutines over `async` / `await` for WebGL compatibility.

Runtime Helpers

- `IndirectHayBuilder.InstallAll()` installs by custom key.
- `IndirectHayBuilder.InjectAll()` injects by custom key and respects the indirect delay config.
- `Haystack.BindDependency<T>()`, `BindStaticDependency<T>()`, `BindColliderDependency<T>()`, and `BindCollider2DDependency<T>()` register dependencies manually.
- `Haystack.TryResolve<T>()`, `TryResolveStatic<T>()`, `Resolve<T>()`, and `ResolveStatic<T>()` fetch dependencies manually.
- `Haystack.Refresh()` rescans direct classes created after scene load.
- `Haystack.Reset()` clears and rebuilds the runtime container.
- `Collider.InjectDependency<T>()` and `Collider2D.InjectDependency<T>()` resolve physics-bound dependencies with optional unbind-after-injection behavior.

Runtime Config

Execution Controls

Runtime Safety

- `AllowStaticClassInjection` enables static installers and injectors.
- `IncludeInactiveObjects` includes disabled scene objects in the direct scan.
- `DirectIterationDelay` adds a small pause between direct scan batches for large scenes.
- `IndirectIterationDelay` delays manual `IndirectHayBuilder` install/inject calls.

- `ResetDependenciesOnSceneLoad` clears tracked dependencies when a scene unloads.
- `CancelPendingOperationsOnSceneUnload` clears queued work on unload.
- `EnableVerboseLogs` enables runtime lifecycle and binding logs.
- `EnableProtection` locks and restores package logic files in the editor by using the hidden protected key stored in the config asset.

Direct Injection

Use direct injection for scene objects that are already present when the scene loads.

```
[HayDirectClass]
public class SampleInstaller : MonoBehaviour
{
    [SerializeField, Install] SampleDependency _sample;

    [HayInstallingMethod]
    protected void Awaker()
    {
        Debug.Log("Dependencies Installed.");
    }
}

[HayDirectClass]
public class SampleInjector : MonoBehaviour
{
    [SerializeField, Inject] SampleDependency _sample;

    [HayInjectedMethod]
    protected void Starter()
    {
        Debug.Log(_sample.Time);
    }
}
```

> If you instantiate a new direct class at runtime, call `Haystack.Refresh()` so the initializer can scan it.

Indirect Builder

Use indirect injection when the dependency source changes at runtime or you want to resolve by custom key.

```
public class CustomPkVariant : MonoBehaviour
{
    [SerializeField] string _customKey;
    [SerializeField, Install] CustomKDependency _dependency;
    readonly IndirectHayBuilder _builder = new IndirectHayBuilder();

    void Awake() => _builder.InstallAll(this, _customKey);
}

public class CustomKManager : MonoBehaviour
{
    [Inject] CustomKDependency _dependency;
    readonly IndirectHayBuilder _builder = new IndirectHayBuilder();

    public void SetToPlayer1()
    {
        _builder.InjectAll(this, () =>
        {
            Debug.Log(_dependency.Name);
        }, "player-1");
    }
}
```

Manual Bind And Resolve

Use manual binding when you want a lightweight container-style workflow without adding installer components.

```
Haystack.BindDependency(new SampleDependency { Time = 12f });

if (Haystack.TryResolve<SampleDependency>(out var dependency))
{
    Debug.Log(dependency.Time);
```

```
}

Haystack.BindStaticDependency("1.0.0", key: "version");
Debug.Log(Haystack.ResolveStatic<string>("version"));
```

Prefer `TryResolve` when a dependency may be absent. `Resolve` returns the default value and logs a warning when nothing is bound.

Sample scripts for this flow are included in `Assets/Haystack/Demo/Sample Demo/InjectionDemo/Assets/Scripts/DemoScripts/ManualBindingSample`.

You can also register collider entries directly at runtime.

```
Haystack.BindColliderDependency(playerCollider, new PlayerDependencies());
Haystack.BindCollider2DDependency(trigger2D, new PlayerDependencies());
```

Collider Injection

Install a dependency against the collider on the same object, then fetch it from overlap, raycast, collision, or trigger callbacks.

```
public class SamplePlayer : MonoBehaviour
{
    [SerializeField, Install(HayComponent.Collider)]
    PlayerDependencies _playerDependencies;

    readonly IndirectHayBuilder _builder = new IndirectHayBuilder();
    void Awake() => _builder.InstallAll(this);
}

[HayPhysicsClass]
public class SampleTrigger : MonoBehaviour
{
    void OnTriggerEnter(Collider other)
    {
        other.InjectDependency<PlayerDependencies>(this, dep =>
        {
            Debug.Log(dep.GameObject.name);
        }, unbindAfterInjection: true);
    }
}
```

```
public class Combat2DEnemy : MonoBehaviour
{
    [Install(HayComponent.Collider2D)]
    Action<int> _takeDamage;
}
```

Static Injection

```
[HayDirectClass]
public static class SampleStatic1
{
    [Install] public static string Dependency;

    [HayInstallingMethod]
    public static void Awaker() => Dependency = "Reckdev";
}

[HayDirectClass]
public static class SampleStaticConsumer
{
    [Inject] public static string Dependency;
}
```

Links

`IHaystackLink` is a lightweight broadcast channel for manual runtime fan-out.

```
public class LinkManager : MonoBehaviour
{
    [SerializeField] LinkDependency _managerDependency;
    void Start() => Haystack.WeLinks.FireLinks(_managerDependency);
}

public class LinkPlayer : MonoBehaviour, IHaystackLink
{
    void Awake() => Haystack.WeLinks.AddLink(this);
    public void Link(object dependency) { }
}
```

Unbinding

Use `HayBind.UnbindAfterInjection` to consume a dependency once after a full injection pass completes.

```
public class SUManager : MonoBehaviour
{
    [Inject(HayBind.UnbindAfterInjection)]
    CustomKDependency _customDependency;
}
```

You can manually clear direct and static registrations at runtime as well.

```
Haystack.Unbind<CustomKDependency>(customKey: playerKey);
Haystack.UnbindStatic<GameConfig>();
```

For collider entries you can manually clear the registration with `collider.Unbind()` or `collider2D.Unbind2D()`, or consume them in one pass by opting into unbind-after-injection.

```
other.InjectDependency<PlayerDependencies>(this, dep =>
{
    Debug.Log(dep.GameObject.name);
}, unbindAfterInjection: true);
```

Unbinding affects the exact binding slot identified by dependency type, inject key, and custom key. Other registrations of the same type remain available unless they share the same slot.

If you consume a slot with `HayBind.UnbindAfterInjection`, install or register that same slot again before expecting another inject to resolve it.

Legacy Aliases

The framework still accepts the old documentation names so older code keeps compiling:

- `[Haystack.Inject]` and `[Haystack.Install]`
- `[Haystack.DirectHayClass]` and `[Haystack.PhysicsClass]`
- `[HayAwaker]` and `[HayStarter]`
- `Haystack.Component.Collider` and `Haystack.ColliderType.Collider3D`
- `Haystack.Bind.UnbindAfterInjection`

Editor Security

When `EnableProtection` is on, Haystack seals a snapshot of the package logic scripts under `Core`, `Runtime`, and `Editor`. If those files are edited locally, the editor restores them from the snapshot during import or compilation. When protection is off, the files remain editable and are not auto-restored.

The protection snapshot uses the hidden `ProtectedKey` from the config asset together with project-bound key material so the package does not rely on a readable plain-text key in the framework scripts.

Package Structure

- `Assets/Haystack/Core/HaystackCore.cs` contains the package registry, binding, and injection core logic.
- `Assets/Haystack/Runtime/Haystack.cs` exposes the public DI API and runtime-facing helpers.
- `Assets/Haystack/Runtime/HaystackInitializer.cs` runs direct/static install and inject passes, including coroutine-returning attributed methods.
- `Assets/Haystack/Resources/HayConfig.asset` stores runtime configuration.
- `Assets/Haystack/Resources/HayEditorSettings.asset` stores editor colors.
- `Assets/Haystack/Resources/HayTM.bytes` stores the encrypted Haystack trademark data used by the custom inspectors and window.
- `Assets/Haystack/Resources/HayGuard.bytes` stores the encrypted package logic snapshot used by editor protection.
- `Assets/Haystack/Demo` contains the packaged sample scenes, demo scripts, and shared demo art.
- `Tools/Haystack/Haystack` opens the Haystack control panel.

Haystack documentation regenerated to match the current runtime API, editor behavior, and packaged demo flows.